

Practical 5: Doubly Linked List and Circular Linked List Operations

➔ Practical Aim

To implement core doubly linked list and circular linked list operations and understand how bidirectional and circular linking change the way nodes are inserted, deleted, and traversed.

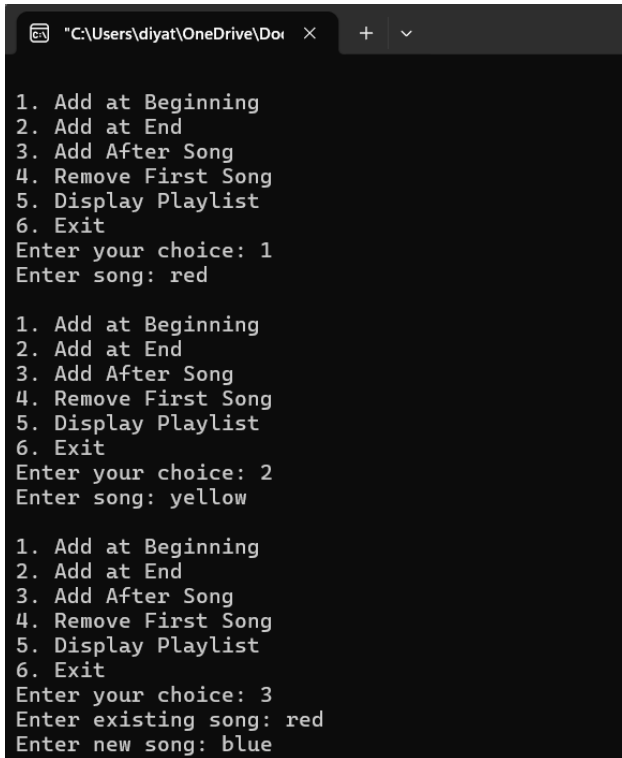
Problem 1

A music player maintains a playlist where songs can be added to the beginning, added to the end, or inserted right after a specific song that is currently playing. When a song is removed, only the first song in the playlist is dropped. At any point the player can count how many songs are in the playlist and display them from first to last. Given a sequence of such operations, implement all of them on a doubly linked playlist and print the result after each operation. Describe how your insertion after a given song works. What extra step does a doubly linked list require here that a singly linked list did not? What happens if the given song is not found in the playlist?

Source Code

https://github.com/diyatolia1212/FDSA-25DCE115_B-Batch/tree/51abea365e86bb7cc796e7336b9d6b6afa36cf2f/PRACTICAL%205

Output



```

1. Add at Beginning
2. Add at End
3. Add After Song
4. Remove First Song
5. Display Playlist
6. Exit
Enter your choice: 1
Enter song: red

1. Add at Beginning
2. Add at End
3. Add After Song
4. Remove First Song
5. Display Playlist
6. Exit
Enter your choice: 2
Enter song: yellow

1. Add at Beginning
2. Add at End
3. Add After Song
4. Remove First Song
5. Display Playlist
6. Exit
Enter your choice: 3
Enter existing song: red
Enter new song: blue

```

s → new song name entered by user.

oldS → the existing song after which you want to add a new song.

- Example: oldS = A, s = C → C will be added after A.

begin() → adds a song at the beginning/front of the playlist.

end() → adds a song at the end/back of the playlist.

after() → adds a new song after a particular existing song.

The program displays the playlist after each operation and shows the total number of songs. If the given song is not found, it displays "Song not found!".

Conclusion

The program successfully implements a doubly linked list playlist with insertion, deletion, counting, and display operations. It also handles insertion after a specific song and shows a message if the song is not found.

Problem 2

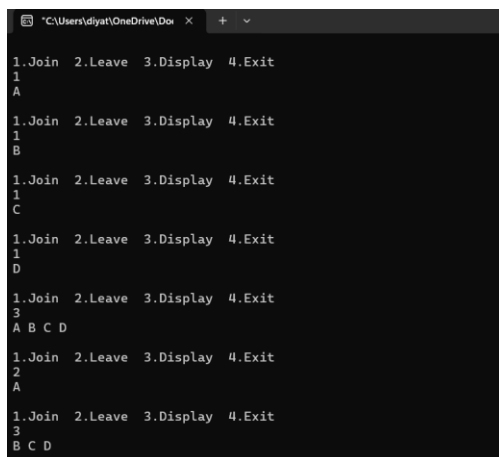
A group of students are sitting in a circle playing a passing game. A token starts at the first student and is passed around the circle one student at a time. Students can join the circle at any position and leave at any time, and the circle must remain unbroken after every join or leave. Given a sequence of join, leave, and display operations, implement them on both a singly circular and a doubly circular linked list and print the current circle after each

operation. Describe how your solution ensures the circle stays unbroken after a student leaves. What is the one case where breaking the circle is most likely to happen as a bug, and how did you handle it?

Source code

https://github.com/diyatolia1212/FDSA-25DCE115_B-Batch/tree/51abea365e86bb7cc796e7336b9d6b6afa36cf2f/PRACTICAL%205

Output



```
*C:\Users\diyat\OneDrive\Doc... X + v
1.Join 2.Leave 3.Display 4.Exit
1
A
1.Join 2.Leave 3.Display 4.Exit
1
B
1.Join 2.Leave 3.Display 4.Exit
1
C
1.Join 2.Leave 3.Display 4.Exit
1
D
1.Join 2.Leave 3.Display 4.Exit
3
A B C D
1.Join 2.Leave 3.Display 4.Exit
2
A
1.Join 2.Leave 3.Display 4.Exit
3
B C D
```

Conclusion

The program successfully implements Doubly Linked List and Circular Linked List with join, leave and display operations.

Challenges Faced

Problem: While deleting the first node in a doubly linked list, the second node may still point to the deleted node.

Solution: I set the prev of the new first node to NULL.

Problem: Circular linked list traversal can go into an infinite loop.

Solution: I stopped traversal when I reached the starting node again.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] A doubly linked list maintains two links per node instead of one. When deleting the front node, explain why having a backward link on the second node creates a responsibility that did not exist in a singly linked list — and what goes wrong if you forget to handle it.

Answer: When the first node is deleted, the second node's prev pointer must be set to NULL. This responsibility does not exist in a singly linked list because it has no prev link. If we forget it, the second node may still point to the deleted node, causing an invalid/dangling pointer and possible errors when traversing backward.

2. [Problem 2] If your traversal of the circular list does not stop at the right moment, it will loop forever. Without referencing any specific code, explain what condition must be true at the moment you stop, and why that condition would fail if you set the last node's link to null.

Answer: In a circular linked list, traversal should stop when you reach the starting (head) node again. This shows that the complete circle has been traversed. If the last node's link is set to NULL, the list is no longer circular, so traversal would stop at NULL instead.